Chorus TF-IDF Song Recommender

The chorus-only model changes the recommendation problem by focusing only on the most repeated and emotionally central part of each song. This reduces noise from verses and narrative detail, but it also removes a large amount of lexical context. The corpus shrinks from 1502 songs in the full-lyrics model to 1051 songs in the chorus corpus, and average token count drops from 145.2 to 50.8. So the chorus model is not simply a cleaner version of TF-IDF; it is operating with less text and on a smaller subset of the original dataset.

1. Imports & Setup

In [1]:
```
import sys
sys.path.append('../src')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from chorus_preprocessing import preprocess_chorus
from chorus_recommender import build_chorus_recommender, recommend_songs
from tfidf_recommender import (
    build_tfidf_recommender,
    recommend_songs as tfidf_recommend
)

pd.set_option('display.max_colwidth', 60)
```

2. Preprocess Chorus Data

In [2]:
```
chorus_df = preprocess_chorus(
    raw_lyrics_path='../data/raw/lyrics.csv',
    output_path='../data/processed/chorus_clean.csv'
)
chorus_df.head()
```

```
Dropped 213 songs flagged as prose/annotation content
Songs kept (chorus found):  1051/1928 (54.5%)
Songs skipped (no chorus):  877/1928 (45.5%)
Saved 1051 songs to ../data/processed/chorus_clean.csv
```

Out[2]:

	track_id	track_name	artist	lyrics	chorus_tex
0	0qNnE0beu4OfHk8Txa3vGt	Soul Flower - Remix	The Pharcyde	[Chorus]\nEe, ah, ooh, ah, ah, ee, ah, ooh, ah, ah\nEe, ...	Ee, ah, ooh, ah ah, ee, ah, ooh, ah ah\nEe, ah, ooh a..
1	7AHX0F99w4x23uSZihklZ6	My God	Tapir!	[Verse 1]\nWaking up with intuition\nGoing back to sleep...	But don't let i brea you\nThere's n complications\nIt..
2	1KbpVvPiLAt1Wsh5gr2YKM	Take a Trip	TV Girl	[Intro]\nTake, take, take a trip, take a trip back down\...	It's not somethin you can poin to\nIt's no something ..
3	56s4IHGnJVGZYcdOVmC3eb	I Love Life, Thank You	Mac Miller	[Intro]\nAlright then, bitch\nHa, ya\nUh\nA million moth...	Ayo, this wher the hook be\nBut ain't got no hook yo..
4	5XufqHphZkJhlkc5wlEST7	Soft Rock Band	Good Morning	[Intro]\nWell I played time in exile\nAnd the weather it...	Oh, I might hav seen it all\nBut know, oh, there' st..

In [3]:

```
# Corpus summary
n_total = len(chorus_df)

print(f"Total songs in chorus corpus: {n_total}")
print()
print("Token count distribution (chorus only):")
print(chorus_df['token_count'].describe().round(1))
```

```
Total songs in chorus corpus: 1051

Token count distribution (chorus only):
count    1051.0
mean       50.8
std        36.6
min        10.0
25%        26.5
50%        41.0
75%        65.0
max       507.0
Name: token_count, dtype: float64
```

3. Chorus Extraction Inspection

In [4]:

```
def inspect_song(track_name, artist=None):
    matches = chorus_df[chorus_df['track_name'].str.lower() == track_name.lo
    if artist:
        matches = matches[matches['artist'].str.lower() == artist.lower()]
```

```
    if matches.empty:
        print(f"'{track_name}' not found.")
        return
    row = matches.iloc[0]
    print(f"Track:       {row['track_name']} — {row['artist']}")
    print(f"Token count: {row['token_count']}")
    print(f"\nRaw chorus text:\n{str(row['chorus_text'])[:600]}")
    print(f"\nProcessed:   {str(row['processed_chorus'])[:300]}")

# Check each of your 4 seed songs
inspect_song('Iron Clad Lou',  'Hum')
print('\n' + '='*70 + '\n')
# Had to switch this seed because it wasn't found in the chorus corpus
inspect_song('Everybody Loves The Sunshine',     'Roy Ayers Ubiquity')
print('\n' + '='*70 + '\n')
# Had to switch this seed because it wasn't found in the chorus corpus
inspect_song('Volare', 'Gipsy Kings')
print('\n' + '='*70 + '\n')
inspect_song('House Money',    'Baby Keem')
print('\n' + '='*70 + '\n')
```

Track: Iron Clad Lou – Hum
Token count: 16

Raw chorus text:
I win, I win, I win
It sure gets lonely at the top
It sure feels weird, you're falling out
It sure gets lonely at the top
I win, I win

Processed: win win win sure get lonely sure feel weird youre falling sure get lonely win win

==========

Track: Everybody Loves The Sunshine – Roy Ayers Ubiquity
Token count: 40

Raw chorus text:
My life, my life, my life, my life
In the sunshine
Everybody loves the sunshine
Sunshine
Everybody loves the sunshine
Sunshine
Folks get down in the sunshine (Oooh)
Sunshine (Yeeea-ah)
Folks get brown in the sunshine
My life, my life, my life, my life
In the sunshine
Everybody loves the sunshine
Sunshine
Everybody loves the sunshine
Sunshine
Folks get down in the sunshine
Sunshine
Folks get brown in the sunshine

Processed: life life life life sunshine everybody love sunshine sunshine everybody love sunshine sunshine folk sunshine oooh sunshine yeeeaah folk brown sunshine life life life life sunshine everybody love sunshine sunshine everybody love sunshine sunshine folk sunshine sunshine folk brown sunshine

==========

Track: Volare – Gipsy Kings
Token count: 32

Raw chorus text:
Vuelve
No volveré, no volveré, no volveré
No quiere recordar
No quiere recordar

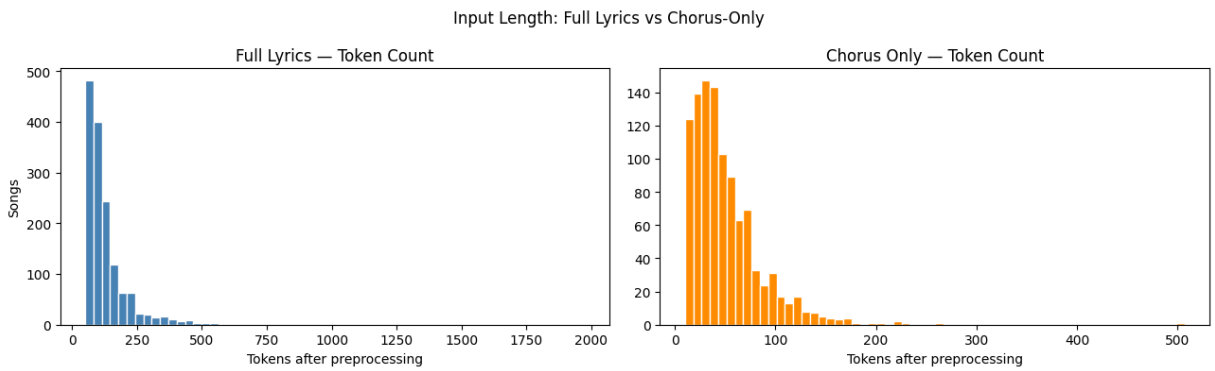
Vuelve
No volveré, no volveré, no volveré

```
No quiere recordar
No quiere recordar
Vuelve
No volveré, no volveré, no volveré
No quiere recordar
No quiere recordar

Vuelve
No volveré, no volveré, no volveré
No quiere recordar
No quiere recordar

Processed:   vuelve volveré volveré volveré quiere recordar quiere recordar
vuelve volveré volveré volveré quiere recordar quiere recordar vuelve volver
é volveré volveré quiere recordar quiere recordar vuelve volveré volveré vol
veré quiere recordar quiere recordar

======================================================================

Track:       House Money - Baby Keem
Token count: 48

Raw chorus text:
Walkin' in the party, I don't fear nobody
What the fuck? Powered up, who gon' stop me?
He a dub, she a bust and I'm godly
Is it fate, is it love? Probably (Whew-whew, whew)
Walkin' in the party, I don't fear nobody
What the fuck? Powered up, who gon' stop me?
He a dub, she a bust and I'm godly
Is it fate, is it love? Probably (Whew-whew, whew)
I smell something (What? What? Pussy, bow-bah-bah-bah, they pussy, what?)
I smell something (Brrt, brrt-brrt-brrt)
I smell something (What? What? Pussy, bow-bah-bah-bah, they pussy, what?)
I smell something (Brrt, brrt-brrt-brrt)

Processed:   walkin party dont fear fuck powered gon stop dub bust im godly
fate love probably whewwhew whew walkin party dont fear fuck powered gon sto
p dub bust im godly fate love probably whewwhew whew smell pussy bowbahbahba
h pussy smell brrt brrtbrrtbrrt smell pussy bowbahbahbah pussy smell brrt br
rtbrrtbrr

======================================================================
```

4. Full Lyrics vs Chorus Only Tokens

In [5]:
```
full_df = pd.read_csv('../data/processed/lyrics_clean.csv')

fig, axes = plt.subplots(1, 2, figsize=(13, 4), sharey=False)

axes[0].hist(full_df['token_count'],   bins=60, color='steelblue',  edgecolo
axes[0].set_title('Full Lyrics — Token Count')
axes[0].set_xlabel('Tokens after preprocessing')
axes[0].set_ylabel('Songs')
```

```
axes[1].hist(chorus_df['token_count'], bins=60, color='darkorange', edgecolc
axes[1].set_title('Chorus Only — Token Count')
axes[1].set_xlabel('Tokens after preprocessing')

plt.suptitle('Input Length: Full Lyrics vs Chorus-Only')
plt.tight_layout()
plt.show()

print("Full lyrics token stats:")
print(full_df['token_count'].describe().round(1))
print("\nChorus-only token stats:")
print(chorus_df['token_count'].describe().round(1))
```




```
Full lyrics token stats:
count    1502.0
mean      145.2
std       181.9
min        50.0
25%        76.0
50%       102.0
75%       147.0
max      1975.0
Name: token_count, dtype: float64

Chorus-only token stats:
count    1051.0
mean       50.8
std        36.6
min        10.0
25%        26.5
50%        41.0
75%        65.0
max       507.0
Name: token_count, dtype: float64
```

A much lower token count for the processed chorus, which is to be expected.

5. Recommenders

In [6]:
```
# Chorus model
chorus_df_model, chorus_tfidf, chorus_sim, chorus_vec = build_chorus_recomme
    '../data/processed/chorus_clean.csv'
```

```
)

# Full-lyric TF-IDF baseline
full_df_model, full_tfidf, full_sim = build_tfidf_recommender(
    '../data/processed/lyrics_clean.csv'
)

print(f"Chorus corpus size:    {len(chorus_df_model)} songs")
print(f"Full lyric corpus:     {len(full_df_model)} songs")
print(f"Chorus TF-IDF matrix:  {chorus_tfidf.shape}")
print(f"Full TF-IDF matrix:    {full_tfidf.shape}")
```

```
Chorus corpus size:    1051 songs
Full lyric corpus:     1502 songs
Chorus TF-IDF matrix:  (1051, 2922)
Full TF-IDF matrix:    (1502, 10000)
```

6. Recommendation Comparison

In [7]:

```
def compare_models(seed_title, seed_artist=None, k=10):
    """Print full-lyric TF-IDF and chorus-only recommendations side by side.
    try:
        full_recs = tfidf_recommend(
            full_df_model, full_sim,
            seed_title=seed_title, seed_artist=seed_artist, k=k
        ).rename(columns={'similarity': 'full_score'})
    except ValueError:
        print(f"[NOT FOUND in full corpus] '{seed_title}'")
        return None, None

    try:
        chorus_recs = recommend_songs(
            chorus_df_model, chorus_sim,
            seed_title=seed_title, seed_artist=seed_artist, k=k
        ).rename(columns={'similarity': 'chorus_score'})
    except ValueError:
        print(f"[NOT FOUND in chorus corpus] '{seed_title}'")
        return None, None

    full_recs.index   += 1
    chorus_recs.index += 1

    print(f"\nSeed: '{seed_title}'{' by ' + seed_artist if seed_artist else
    print(f"{'Rank':<5} {'── Full Lyrics ──':<46} {'── Chorus Only ──':<46}"
    print('-' * 97)
    for i in range(1, k + 1):
        f = full_recs.loc[i]
        c = chorus_recs.loc[i]
        f_str = f"{f['track_name'][:28]} — {f['artist'][:12]} ({f['full_scor
        c_str = f"{c['track_name'][:28]} — {c['artist'][:12]} ({c['chorus_sc
        print(f"{i:<5} {f_str:<46} {c_str:<46}")

    return full_recs, chorus_recs
```

In [8]:
```
# --- Grunge ---
full_grunge, chorus_grunge = compare_models('Iron Clad Lou', seed_artist='Hu
```

```
Seed: 'Iron Clad Lou' by Hum
Rank  —— Full Lyrics ——                                  —— Chorus Only ——
--------------------------------------------------------------------------------
---------------------
1     Stand up and Win — Spymob (0.144)                  Faxing It In — Smushie
(0.248)
2     Never Lonely — Space Mounta (0.135)                The Great Salt Lake — B
and of Hors (0.199)
3     Downtown — Varsity (0.123)                         Belief — John Mayer (0.
194)
4     Rain — World's Expe (0.119)                        Weird Fishes / Arpeggi
— Radiohead (0.183)
5     Faxing It In — Smushie (0.112)                     There’d Better Be A Mir
rorba — Arctic Monke (0.173)
6     Hippies — flipturn (0.109)                         UFO — The Juice Is (0.1
71)
7     Warmest Part of the Winter — Voxtrot (0.106)       Arabella — Arctic Monke
(0.168)
8     2023 — Blue Rain Bo (0.105)                        Alright — Supergrass
(0.154)
9     Alive — Pearl Jam (0.103)                          Cherry Blossom — Raury
(0.152)
10    Down the Line — Beach Fossil (0.090)               The Bottom Shelf — Dani
el Boxx (0.141)
```

Similarity scores are higher on averarge in the chorus only version, but not a single song on the list is a grunge song. This further emphasizes the point that grunge music is better defined by a hardcore rock sound than the lyrical message.

In [ ]:
```
# --- Soul ---
full_soul, chorus_soul = compare_models(
    'Everybody Loves The Sunshine', seed_artist= 'Roy Ayers Ubiquity')
```

```
Seed: 'Everybody Loves The Sunshine' by Roy Ayers Ubiquity
Rank  —— Full Lyrics ——                                —— Chorus Only ——
--------------------------------------------------------------------------------
---------------------
1     Holy Moly, Rock n' Rolly, Gu — LAZY GHOST (0.328) Sunshine Of Your Lov
e — Cream (0.225)
2     Sunshine Of Your Love — Cream (0.153)          Strawberry Letter 23 —
Shuggie Otis (0.212)
3     Sir Duke — Stevie Wonde (0.149)                You Only Live Once — Th
e Strokes (0.189)
4     Ain't No Sunshine — Bill Withers (0.137)       Sunday Best — Surfaces
(0.179)
5     Sunday Best — Surfaces (0.130)                 Maybelle — Ida (0.152)
6     You And Your Folks, Me And M — Funkadelic (0.129) Hesitation — Hot Fla
sh He (0.132)
7     Sunshine — The Velvet C (0.127)                Mr. Jones — Counting Cr
o (0.128)
8     You Only Live Once — The Strokes (0.122)       Ventura Highway — Ameri
ca (0.127)
9     Why Would I? — nelward (0.117)                 Best Tears — The Happy
Fi (0.126)
10    Brakhage — Stereolab (0.113)                   You And Your Folks, Me
And M — Funkadelic (0.117)
```

Like in the grunge test, the chorus only similarity scores were higher on average. However, only one soul song makes the list: Starberry Letter 23. The full lyric recommender found two soul songs: "Sir Duke" and "Ain't No Sunshine." Overall, both recommenders are sturggling with soul music.

In [21]:

```python
# --- Latin ---
full_latin, chorus_latin = compare_models(
    'Heaven Knows', seed_artist= '54 Ultra'
)
```

```
Seed: 'Heaven Knows' by 54 Ultra
Rank  —— Full Lyrics ——                                —— Chorus Only ——
--------------------------------------------------------------------------------
---------------------
1     A Dozen Roses — Braid (0.334)                    Lapdance — N.E.R.D (0.3
35)
2     Sweetest Thing on This Side  — Papa Bear &  (0.309) Ann Don't Cry — Pa
vement (0.314)
3     Pennies From Heaven — Louis Prima (0.260)       Jerk It Out — Caesars
(0.257)
4     The Scoop On Heaven — Mac Miller (0.256)        Cause = Time — Broken S
ocia (0.252)
5     For Martha — The Smashing (0.247)               You Only Live Once — Th
e Strokes (0.220)
6     Where Are You — 54 Ultra (0.194)                Fade — Knowmadic (0.21
5)
7     Cool God — Jack Symes (0.176)                   I Don't Mind — The Shed
(0.202)
8     Take a Trip — TV Girl (0.172)                   Bound For The Floor — L
ocal H (0.202)
9     A Cold Sunday — Lil Yachty (0.169)              Prom Song — Surfer Bloo
d (0.200)
10    Cadence — The Long Fac (0.141)                  Dance With Me — Sir, Pl
ease (0.198)
```

The seed song for Latin music had to change, because the original see was removed in chorus preprocessing. For this seed, both recomemnders performed quite similar but the chorus only recommender did better on average. Yet the same lack of genre representation seen in the first two tests carries over here. Both full lyric model and chorus model had zero latin represnetation in its top-10 most semantically similar songs.

In [22]:
```python
# --- Hip-hop ---
full_hiphop, chorus_hiphop = compare_models(
    'House Money', seed_artist='Baby Keem')
```

```
Seed: 'House Money' by Baby Keem
Rank  — Full Lyrics —                                  — Chorus Only —
--------------------------------------------------------------------------------
---------------------
1     Jesus Is The One (I Got Depr — Zack Fox (0.441) Blood On Me — Sampha
(0.235)
2     Can You Rap Like Me? — Trippie Redd (0.363)    The Killing Moon — Echo
& the B (0.202)
3     Hit Bout It — Lil Yachty (0.329)               Tried And True — Ween
(0.201)
4     Blood On Me — Sampha (0.321)                   fafo — Zack Fox (0.181)
5     REEL IT IN — Aminé (0.286)                     I Love Life, Thank You
— Mac Miller (0.167)
6     trademark usa — Baby Keem (0.272)              Watermelon — John + Jan
e  (0.146)
7     JayJay-Z — Zelooperz (0.266)                   Stop Whispering — Radio
head (0.133)
8     Fresh Prince Of Utah — YoungBoy Nev (0.262)    (Don't Fear) The Reaper
— Blue Öyster  (0.125)
9     CORSO — Tyler, The C (0.260)                   Funny Thing — Thunderca
t (0.115)
10    WILSHIRE — Tyler, The C (0.237)                Black Qualls — Thunderc
at (0.112)
```

Finally an interesting result from the chorus model! Surprisingly, the chorus only recommender performed significantly worse than the full lyric recommender. The full lyric model recommendations are all in the hip-hop genre, whereas the chorus only recommender only has five hip-hop recommendations. I hypothesize that the reason the chrous recommender is worse at recognizing hip-hop songs is because the similarity found in the full lyric model is primarily due to adlibs and explicative not found frequently in the chorus. Choruses are the heart of a song's message, and based on the chorus recommender, we can see that those messages vary.

7. Overlap: Full Lyrics vs Chorus

In [23]:

```python
def overlap_stats(full_recs, chorus_recs, label):
    if full_recs is None or chorus_recs is None:
        print(f"{label}: skipped (not found)\n")
        return
    full_set   = set(zip(full_recs['track_name'],   full_recs['artist']))
    chorus_set = set(zip(chorus_recs['track_name'], chorus_recs['artist']))
    shared   = full_set & chorus_set
    jaccard  = len(shared) / len(full_set | chorus_set)
    print(f"{label}")
    print(f"  Shared songs:   {len(shared)}/10")
    print(f"  Jaccard score:  {jaccard:.2f}")
    if shared:
        print(f"  Overlap:        {', '.join([s[0] for s in shared])}")
    print()

overlap_stats(full_grunge,  chorus_grunge,  'Grunge  (Iron Clad Lou)')
overlap_stats(full_soul,    chorus_soul,    'Soul    (Everybody Loves the Su
```

```
overlap_stats(full_latin,   chorus_latin,   'Latin   (Heaven Knows)')
overlap_stats(full_hiphop,  chorus_hiphop,  'Hip-hop (House Money)')
```

```
Grunge  (Iron Clad Lou)
  Shared songs:   1/10
  Jaccard score:  0.05
  Overlap:        Faxing It In

Soul    (Everybody Loves the Sunshine)
  Shared songs:   4/10
  Jaccard score:  0.25
  Overlap:        Sunshine Of Your Love, You Only Live Once, Sunday Best, Yo
u And Your Folks, Me And My Folks

Latin   (Heaven Knows)
  Shared songs:   0/10
  Jaccard score:  0.00

Hip-hop (House Money)
  Shared songs:   1/10
  Jaccard score:  0.05
  Overlap:        Blood On Me
```

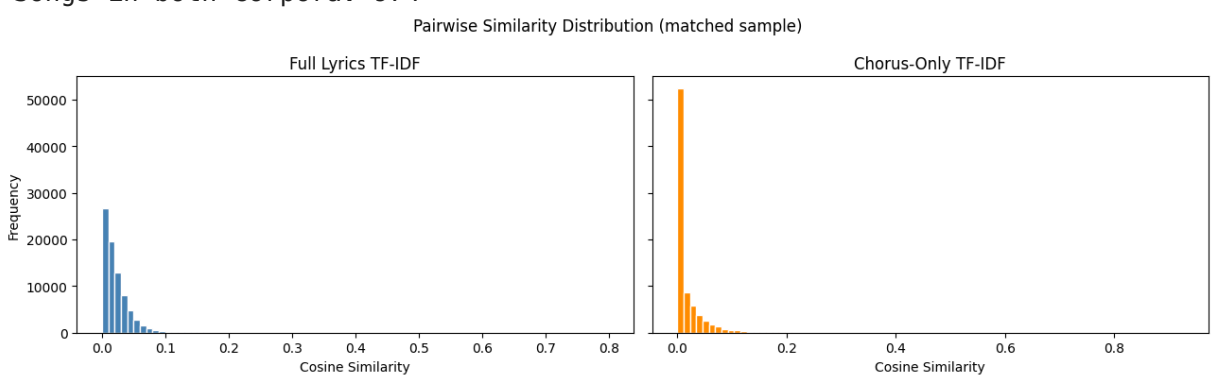
The very low overlap between full-lyric and chorus-only top-10 lists shows that these two models are capturing different definitions of similarity. Full lyrics reflect broader song composition and vocabulary, while chorus-only text isolates the repeated lyrical hook. Neither is universally better; they emphasize different parts of a song's identity.

8. Similarity Score Distribution: Full vs Chorus

In [24]:
```
# Use only songs present in both corpora for a fair comparison
shared_songs = set(zip(full_df_model['track_name'], full_df_model['artist']))
               set(zip(chorus_df_model['track_name'], chorus_df_model['artis
print(f"Songs in both corpora: {len(shared_songs)}")

sample_size = min(400, len(shared_songs))
rng = np.random.default_rng(42)

# Get indices in each DataFrame for the shared sample
shared_list = list(shared_songs)
sample_songs = [shared_list[i] for i in rng.choice(len(shared_list), sample_

def get_indices(df, song_list):
    idx = []
    for name, artist in song_list:
        match = df[(df['track_name'] == name) & (df['artist'] == artist)]
        if not match.empty:
            idx.append(match.index[0])
    return idx

full_idx   = get_indices(full_df_model,   sample_songs)
chorus_idx = get_indices(chorus_df_model, sample_songs)
```

```
n = min(len(full_idx), len(chorus_idx))
full_sample   = full_sim[np.ix_(full_idx[:n],   full_idx[:n])]
chorus_sample = chorus_sim[np.ix_(chorus_idx[:n], chorus_idx[:n])]

full_upper   = full_sample[np.triu_indices(n, k=1)]
chorus_upper = chorus_sample[np.triu_indices(n, k=1)]

fig, axes = plt.subplots(1, 2, figsize=(13, 4), sharey=True)
axes[0].hist(full_upper,   bins=80, color='steelblue',  edgecolor='white')
axes[0].set_title('Full Lyrics TF-IDF')
axes[0].set_xlabel('Cosine Similarity')
axes[0].set_ylabel('Frequency')

axes[1].hist(chorus_upper, bins=80, color='darkorange', edgecolor='white')
axes[1].set_title('Chorus-Only TF-IDF')
axes[1].set_xlabel('Cosine Similarity')

plt.suptitle('Pairwise Similarity Distribution (matched sample)')
plt.tight_layout()
plt.show()

for label, arr in [('Full lyrics', full_upper), ('Chorus only', chorus_upper
    print(f"{label}:  mean={arr.mean():.4f}  median={np.median(arr):.4f}  "
          f">0.1: {(arr > 0.1).mean()*100:.1f}%  >0.3: {(arr > 0.3).mean()*1
```

```
Songs in both corpora: 974
```




```
Full lyrics:  mean=0.0226  median=0.0162  >0.1: 1.2%  >0.3: 0.1%
Chorus only:  mean=0.0168  median=0.0000  >0.1: 2.8%  >0.3: 0.1%
```

This is one of the most interesting results in the notebook. The chorus-only model has a lower mean similarity overall than the full-lyrics model, but a slightly larger share of pairs above 0.1. That suggests chorus-only similarity is more polarized: many song pairs share almost nothing once verses are removed, but a smaller subset becomes more strongly connected through repeated hook language.

9. Observations & Notes

The chorus-only experiment ended up being more useful than I expected because it showed that “core message” similarity is not the same as full-song similarity. By stripping away verses, the model sometimes produced stronger top matches for songs whose emotional hook is especially central and repetitive. But it also lost a lot of

contextual information, which made the corpus smaller and the average input much shorter. That reduction in information helps explain why the overall mean similarity was actually lower than in the full-lyrics TF-IDF model, even though some seed-song recommendations looked more convincing.

The biggest takeaway is that chorus text and full lyrics capture different dimensions of similarity. Full lyrics better represent a song's total lexical composition, while chorus-only text emphasizes the repeated hook, emotional center, and memorable phrasing. The low overlap between the two models confirms that these are not interchangeable approaches. For this project, the chorus recommender is best understood not as a replacement for full TF-IDF, but as an alternative lens on what lyrical similarity means.